Gathering the herd
A history of population control in distributed systems

Corto Mascle

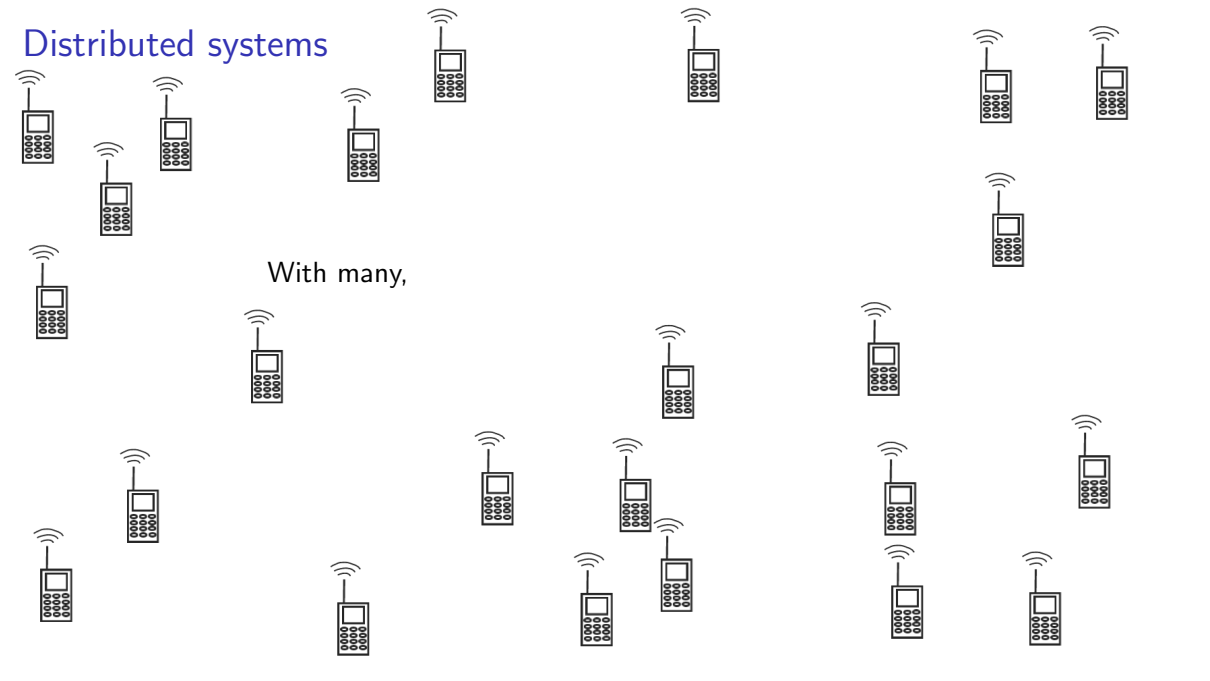
Max Planck Institute for Software Systems

GT Vérif 2026

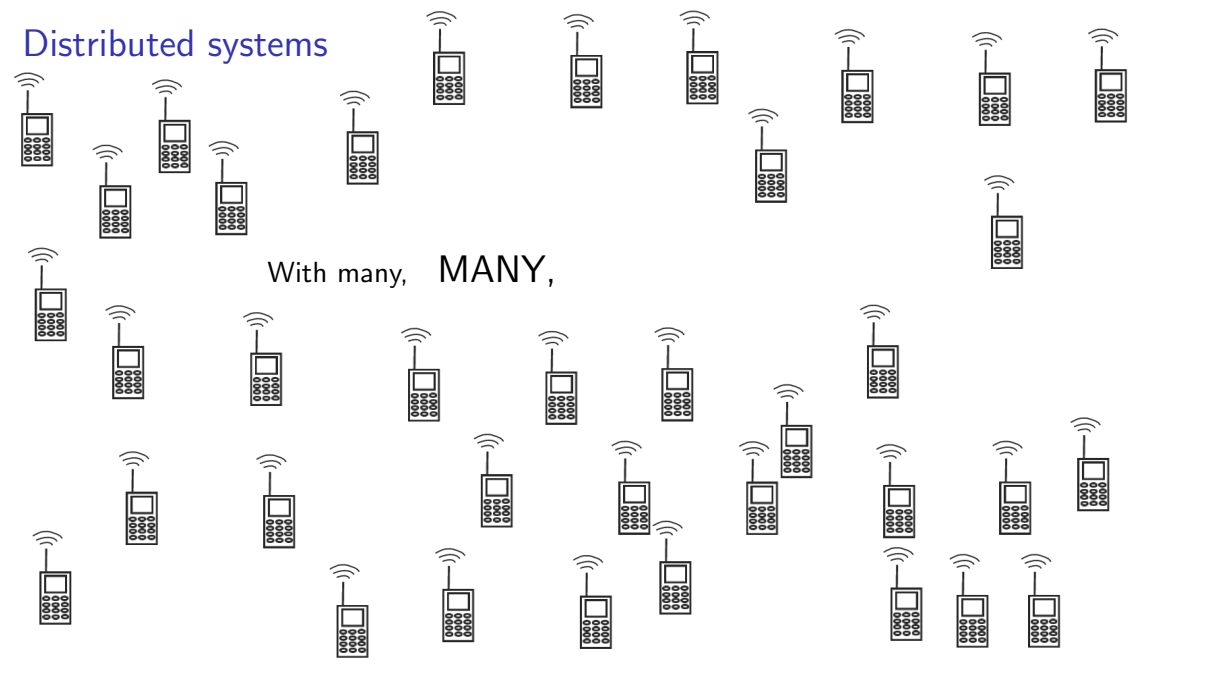
Distributed systems

Distributed systems

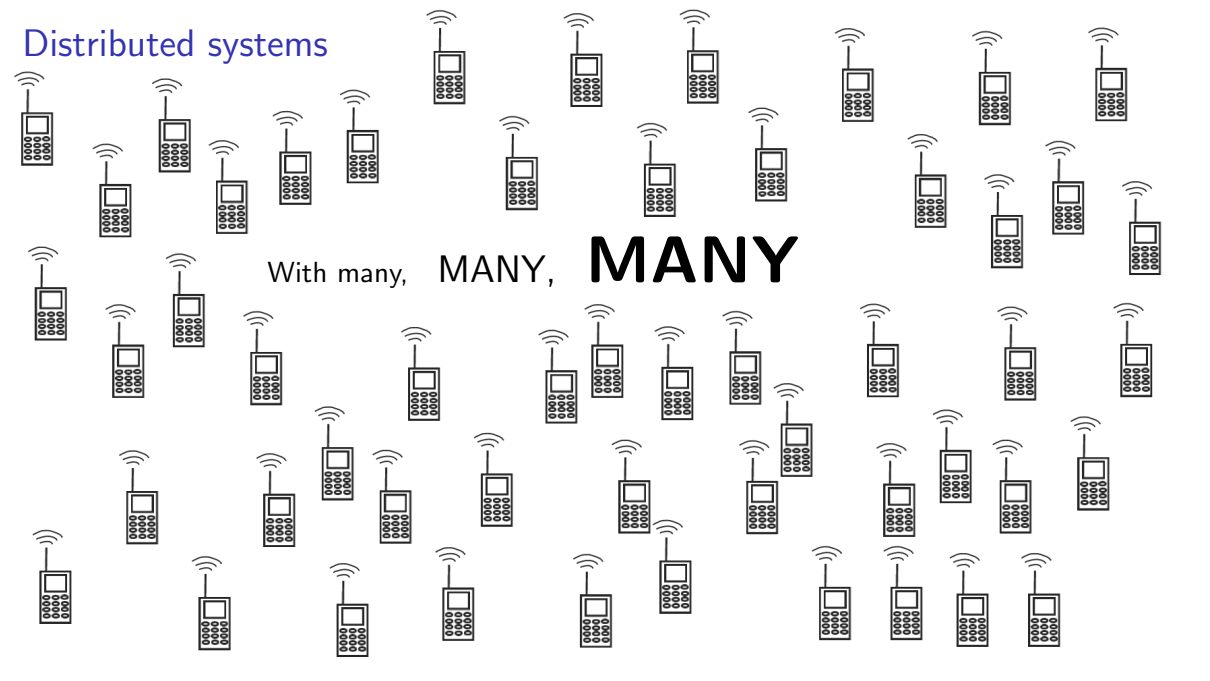
With many,



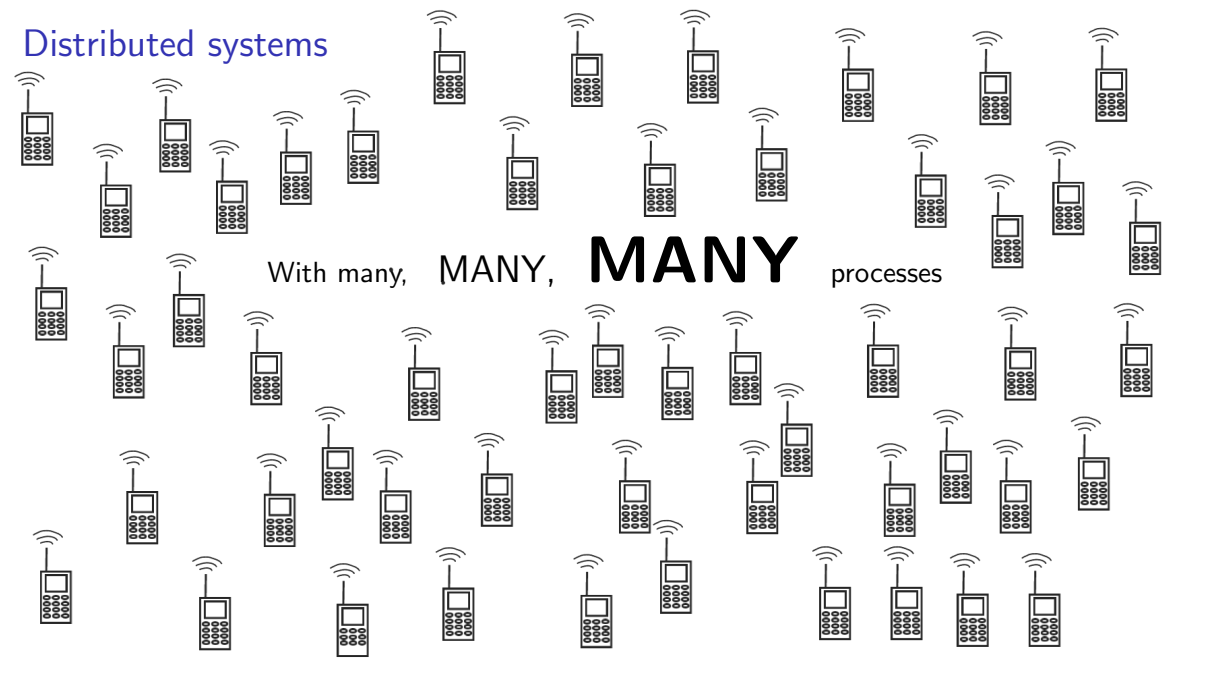
Distributed systems



Distributed systems



Distributed systems



Distributed systems

With many, MANY, **MANY** processes

Multi-threaded programs

Distributed systems

With many, MANY, **MANY** processes

Multi-threaded programs

Client–server systems

Distributed systems

With many, MANY, **MANY** processes



Distributed systems

With many, MANY, **MANY** processes



Distributed systems

With many, MANY, **MANY** processes



Distributed systems

Distributed systems

Parameterized models

An arbitrary number of identical agents

Ad-hoc networks, rendez-vous protocols, ...

Distributed systems

Parameterized models

An arbitrary number of identical agents

Ad-hoc networks, rendez-vous protocols, ...

Dynamic models

Number of processes may increase during runtime

Multi-threaded programs, Petri nets, ...

Distributed systems

Parameterized models

An arbitrary number of identical agents

Ad-hoc networks, rendez-vous protocols, ...

This talk

Dynamic models

Number of processes may increase during runtime

Multi-threaded programs, Petri nets, ...

Parameterised verification

Given a process P, is the system of N copies of P correct

for every population size N?

Parameterised verification

Given a process P, is the system of N copies of P correct

for every population size N?

Typical systems

- Broadcast networks
- Token passing
- Mutexes
- Shared variables

Typical objectives

Safety e.g. deadlock-freeness

No bad configuration is reachable.

Liveness e.g. termination

A good behaviour eventually occurs.

Fair termination e.g. consensus

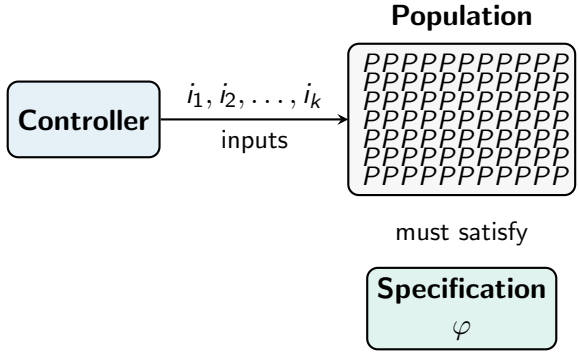
The system converges to a desired state.

Control problem

We have a system made of many identical processes.

Control problem

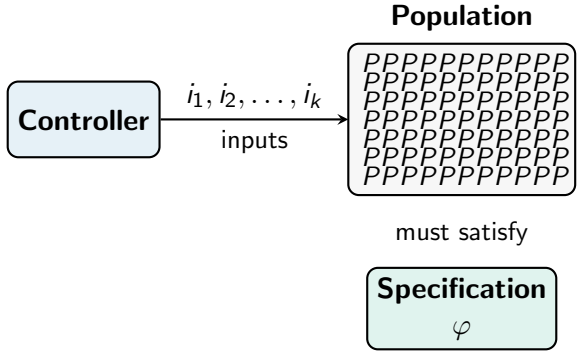
We have a system made of many identical processes.



We want a **controller** that can steer the **population** of processes to meet a given **specification**.

Control problem

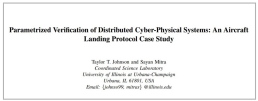
We have a system made of many identical processes.



We want a **controller** that can steer the **population** of processes to meet a given **specification**.

Motivation

Cyber-physical systems



Parametrized Verification of Distributed Cyber-Physical Systems: An Aircraft Landing Protocol Case Study

Taylor T. Johnson and Sayan Mitra
Coordinated Science Laboratory
University of Illinois at Urbana-Champaign
Urbana, IL 61801, USA
Email: {johnso49, mitras}@illinois.edu

Motivation

Cyber-physical systems



Parametrized Verification of Distributed Cyber-Physical Systems: An Aircraft Landing Protocol Case Study

Taylor T. Johnson and Sayan Mitra
Coordinated Science Laboratory
University of Illinois at Urbana-Champaign
Urbana, IL 61801, USA
Email: {johnso49, mitras}@illinois.edu

Traffic regulation

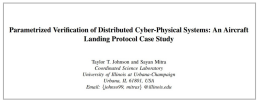


Max pressure control of a network of signalized intersections

Pravin Varaiya *
Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, CA 94720-1770, United States

Motivation

Cyber-physical systems



Parametrized Verification of Distributed Cyber-Physical Systems: An Aircraft Landing Protocol Case Study

Taylor T. Johnson and Sayan Mitra
Coordinated Science Laboratory
University of Illinois at Urbana-Champaign
Urbana, IL 61801, USA
Email: {johnso99, mitras}@illinois.edu

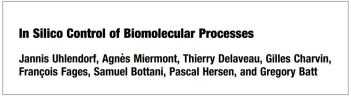
Traffic regulation



Max pressure control of a network of signalized intersections

Pravin Varaiya *
Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, CA 94720-1770, United States

Biochemical systems



In Silico Control of Biomolecular Processes

Jannis Uhlendorf, Agnès Miermont, Thierry Delaveau, Gilles Charvin, François Fages, Samuel Bottani, Pascal Hersen, and Gregory Batt

Motivation

Cyber-physical systems



Parametrized Verification of Distributed Cyber-Physical Systems: An Aircraft Landing Protocol Case Study

Taylor T. Johnson and Sayan Mitra
Coordinated Science Laboratory
University of Illinois at Urbana-Champaign
Urbana, IL 61801, USA
Email: {johnso96, mitras}@illinois.edu

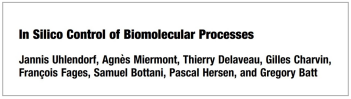
Traffic regulation



Max pressure control of a network of signalized intersections

Pravin Varaiya *
Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, CA 94720-1770, United States

Biochemical systems



In Silico Control of Biomolecular Processes

Jannis Uhlendorf, Agnès Miermont, Thierry Delaveau, Gilles Charvin, François Fages, Samuel Bottani, Pascal Hersen, and Gregory Batt

Sociological studies

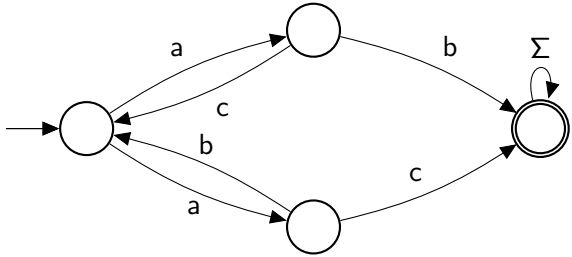


OPEN **Modelling opinion dynamics under the impact of influencer and media strategies**

Lucia Halfmann[1,2], Natalia Djurdjevac Conrad[1], Philipp Lorenz-Spreen[3] & Christof Schütte[1,4]✉

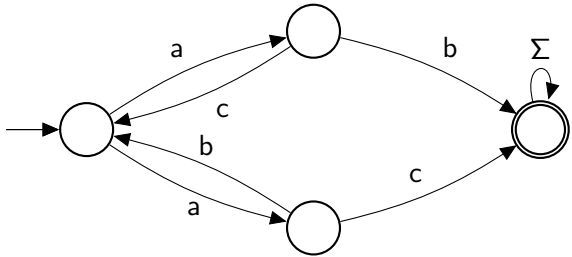
Adversarial population control [Bertrand, Dewaskar, Genest, Gimbert 2017]

$\mathcal{A}$ an NFA (= 1 process). Two players: **Controller** and **Opponent**.



Adversarial population control [Bertrand, Dewaskar, Genest, Gimbert 2017]

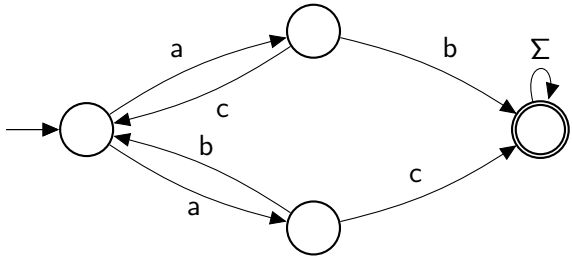
$\mathcal{A}$ an NFA ($= 1$ process). Two players: **Controller** and **Opponent**.



- We start with N processes in the initial state.

Adversarial population control [Bertrand, Dewaskar, Genest, Gimbert 2017]

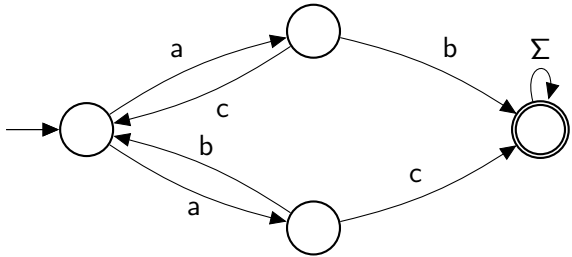
$\mathcal{A}$ an NFA ($= 1$ process). Two players: **Controller** and **Opponent**.



- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$

Adversarial population control [Bertrand, Dewaskar, Genest, Gimbert 2017]

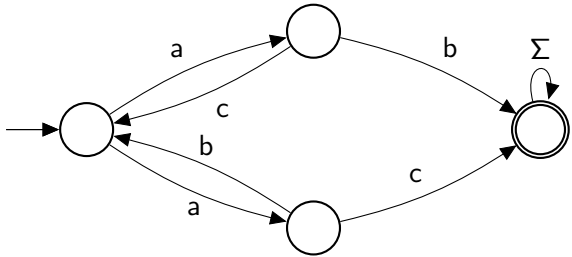
$\mathcal{A}$ an NFA ($= 1$ process). Two players: **Controller** and **Opponent**.



- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$
  - Opponent chooses an x-transition for each process.

Adversarial population control [Bertrand, Dewaskar, Genest, Gimbert 2017]

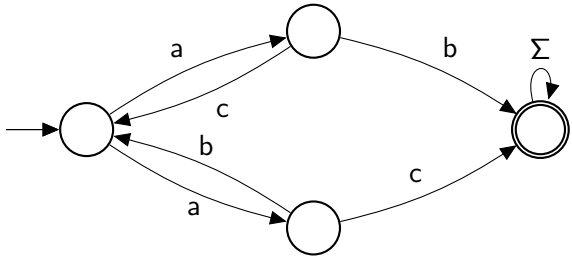
$\mathcal{A}$ an NFA ($= 1$ process). Two players: **Controller** and **Opponent**.



- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$
  - Opponent chooses an x-transition for each process.
- Controller wins when all processes are in the final state.

Adversarial population control [Bertrand, Dewaskar, Genest, Gimbert 2017]

$\mathcal{A}$ an NFA ($= 1$ process). Two players: **Controller** and **Opponent**.



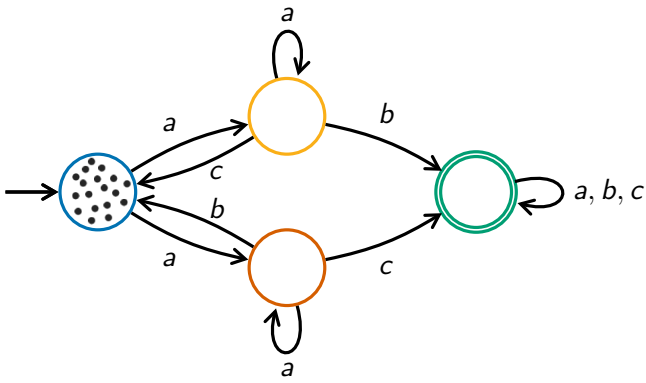
- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$
  - Opponent chooses an x-transition for each process.
- Controller wins when all processes are in the final state.

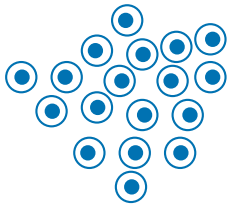
Population control problem

Input: an NFA $\mathcal{A}$

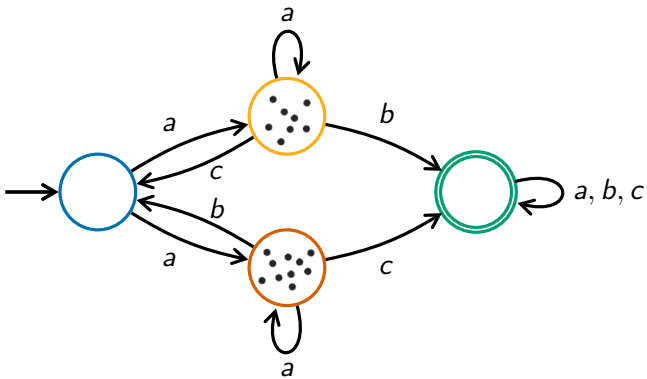
Output: Can Controller win over $\mathcal{A}$ for all N?

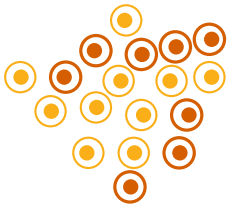
Adversarial population control



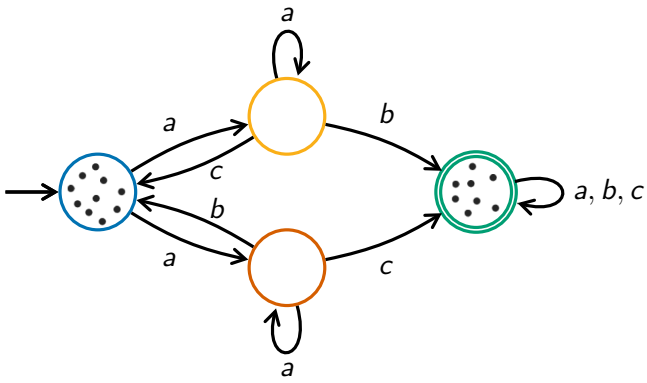


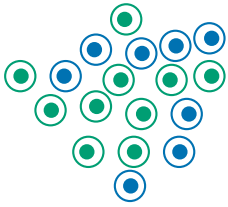
Adversarial population control



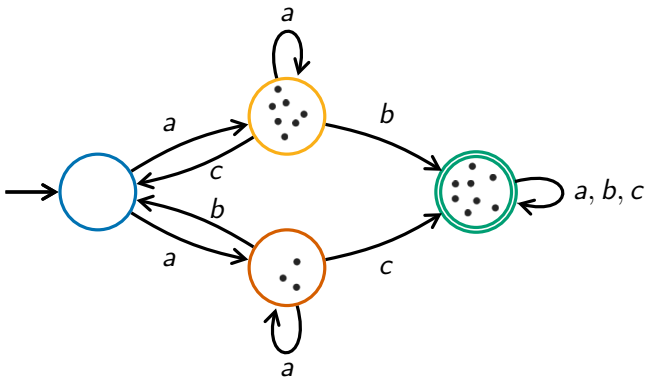


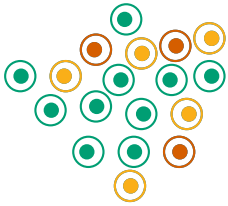
Adversarial population control



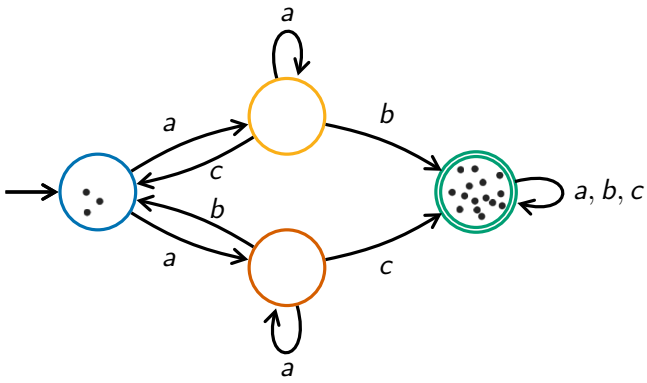


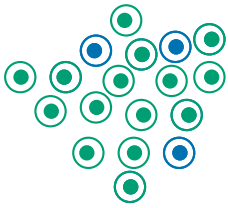
Adversarial population control



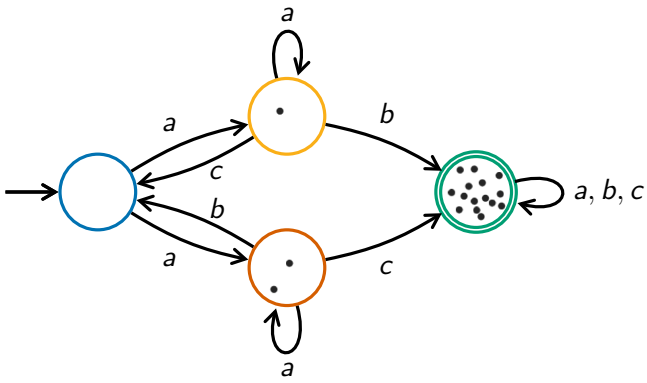


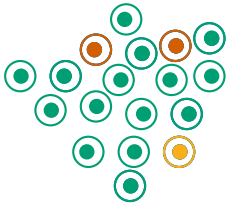
Adversarial population control



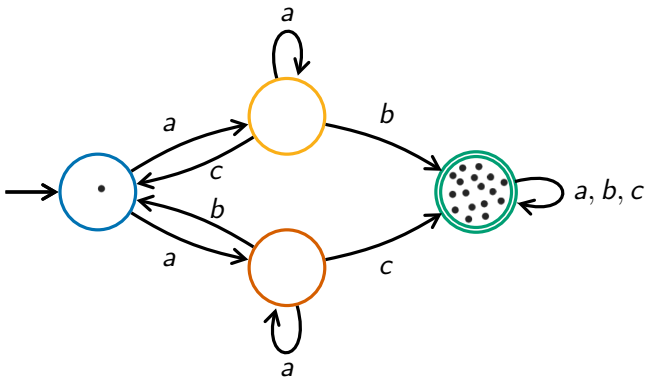


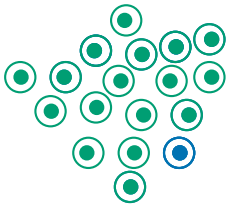
Adversarial population control



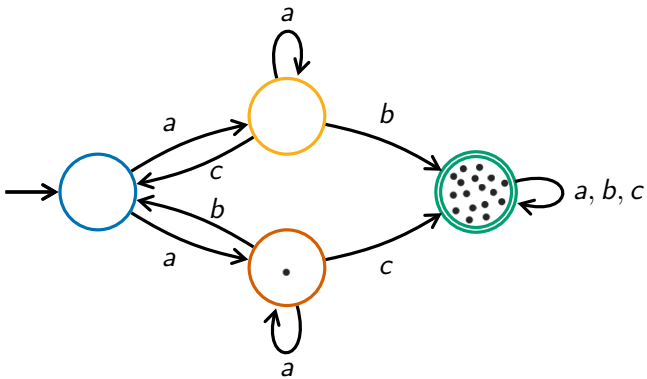


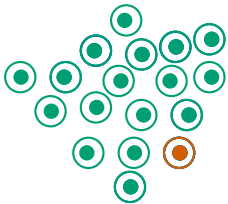
Adversarial population control



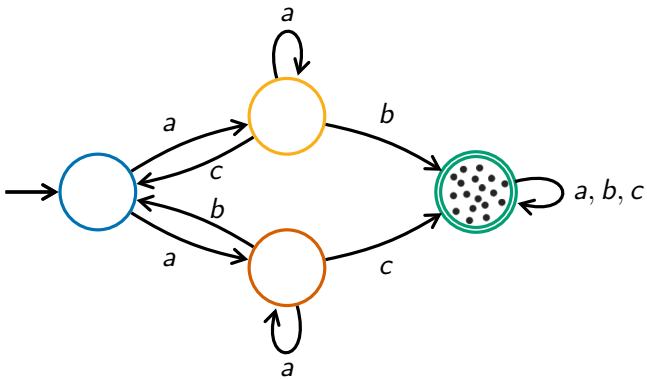


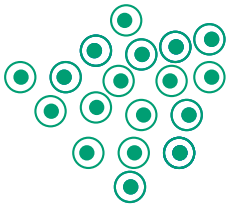
Adversarial population control



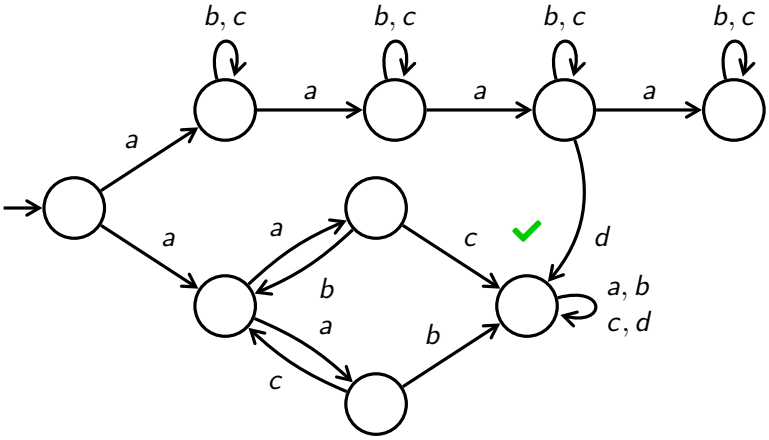


Adversarial population control

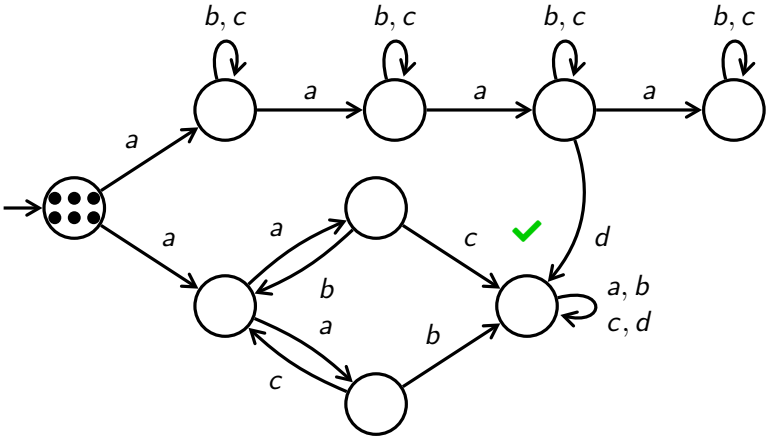




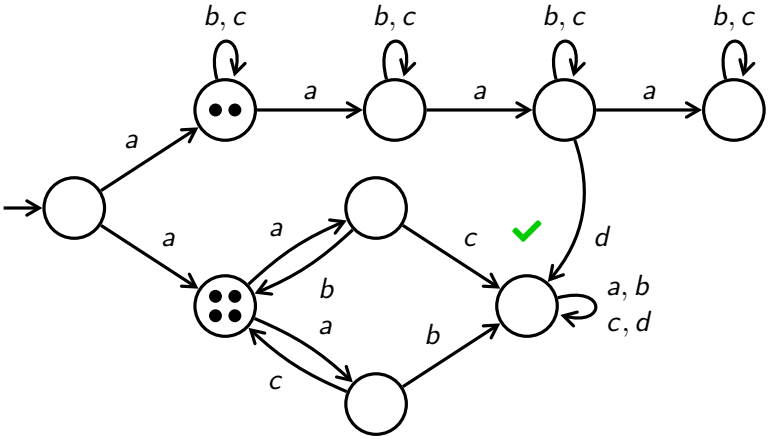
Adversarial population control



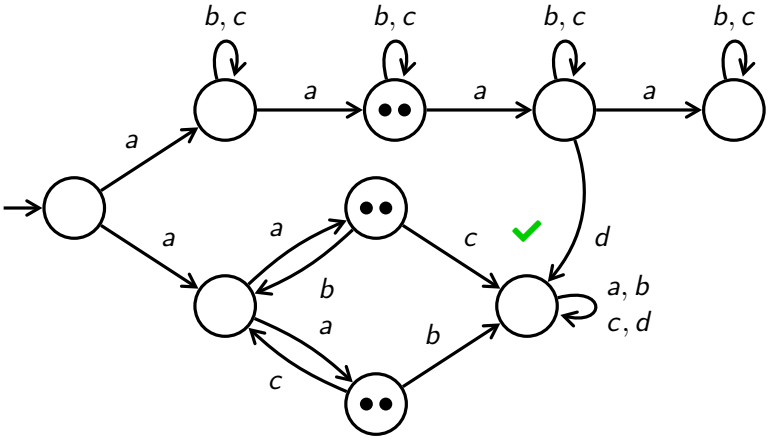
Adversarial population control



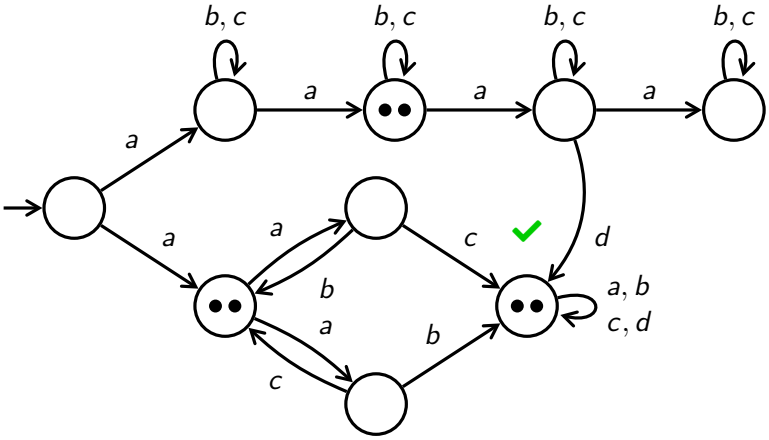
Adversarial population control



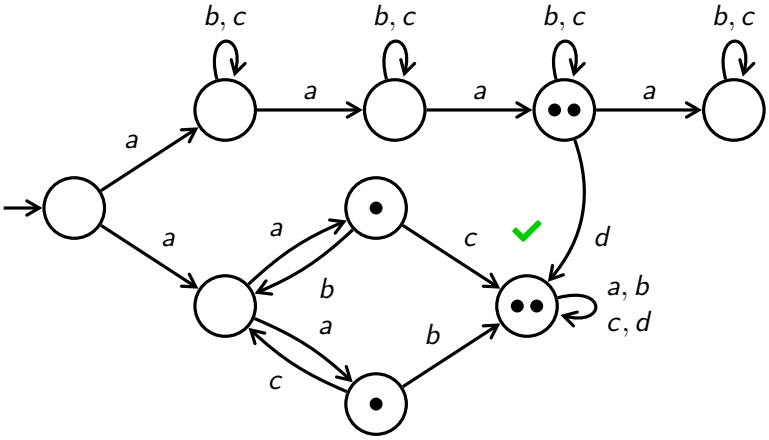
Adversarial population control



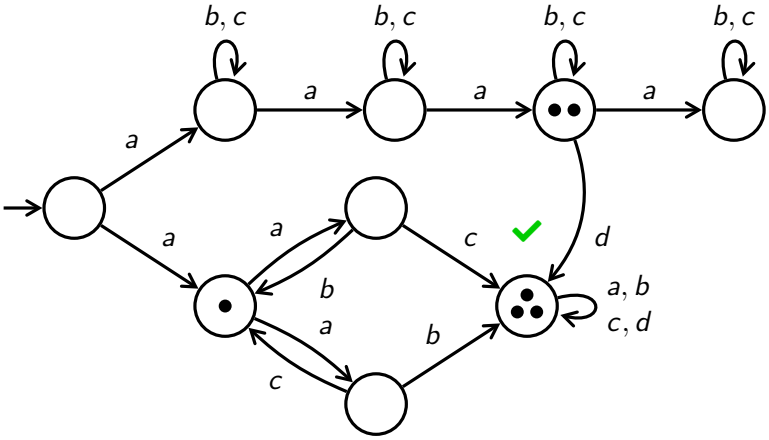
Adversarial population control



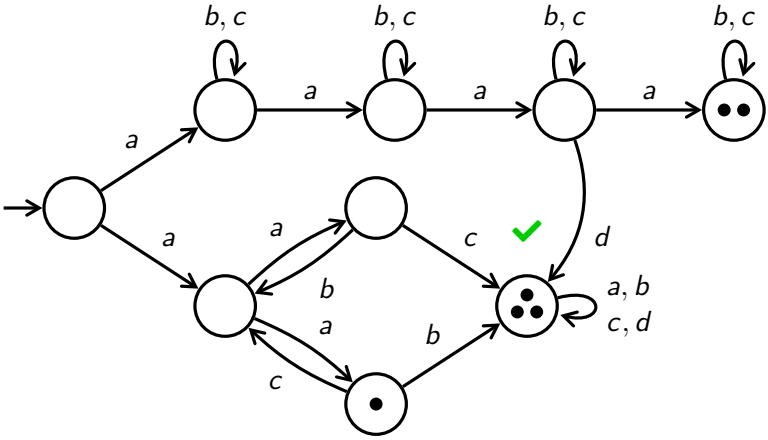
Adversarial population control



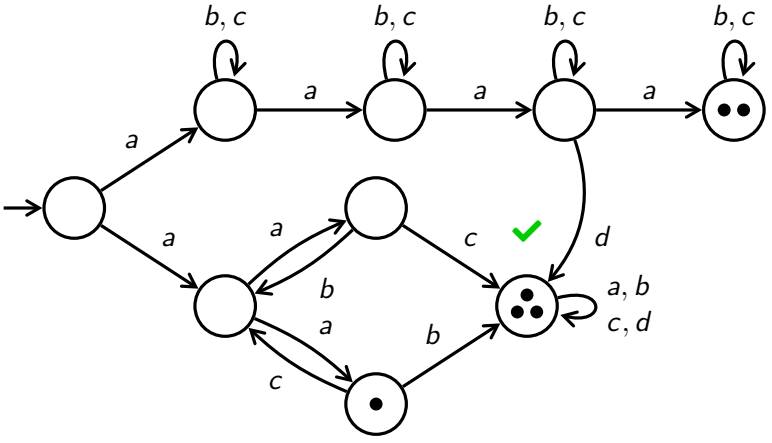
Adversarial population control



Adversarial population control



Adversarial population control



Adapted from [Bertrand Dewaskar Genest Gimbert '15]

There are MDPs of size $\mathcal{O}(k)$ for which we can win against 2^{2^k} tokens and not more.

Adversarial population control

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

The Control Problem is decidable for populations of NFAs.

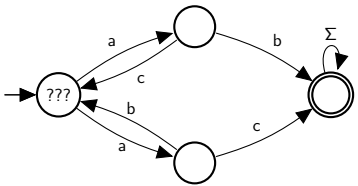
Adversarial population control

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

The Control Problem is decidable for populations of NFAs.

Proof:

Hide the number of tokens!



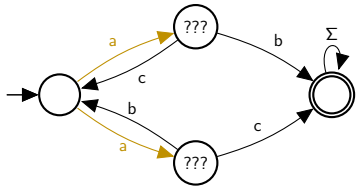
Adversarial population control

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

The Control Problem is decidable for populations of NFAs.

Proof:

Hide the number of tokens!



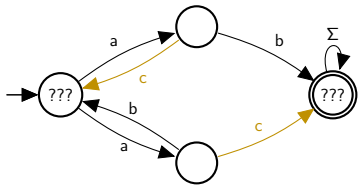
Adversarial population control

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

The Control Problem is decidable for populations of NFAs.

Proof:

Hide the number of tokens!



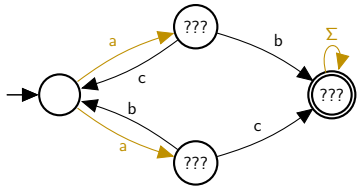
Adversarial population control

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

The Control Problem is decidable for populations of NFAs.

Proof:

Hide the number of tokens!



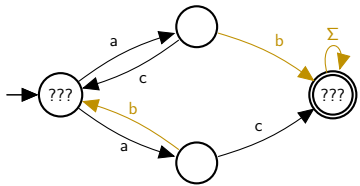
Adversarial population control

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

The Control Problem is decidable for populations of NFAs.

Proof:

Hide the number of tokens!



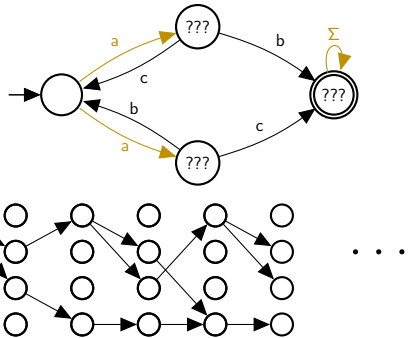
Adversarial population control

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

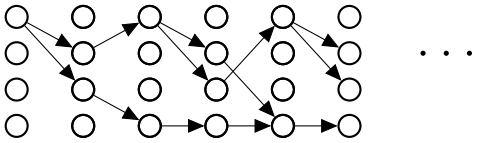
The Control Problem is decidable for populations of NFAs.

Proof:

Hide the number of tokens!

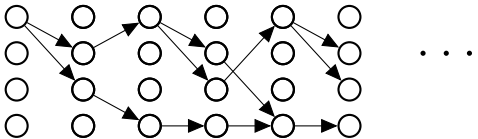


Checking the winning condition



. . .

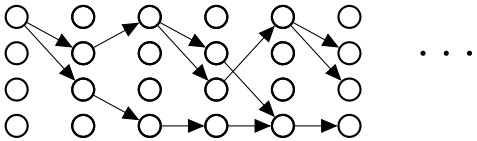
Checking the winning condition



Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

There is a finite parity automaton which checks whether a play is feasible with finitely many tokens.

Checking the winning condition



...

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

There is a finite parity automaton which checks whether a play is feasible with finitely many tokens.

We can construct a finite game such that Controller wins in the game if and only if she has a population control strategy for all N.

Theorem [Bertrand, Dewaskar, Genest, Gimbert 2017]

The Adversarial Population Control Problem is EXPTIME-complete.

So far...

- We have a model for populations of finite-state processes with reachability objectives
- We can decide if a controller exists for such a population!

HOWEVER

So far...

- We have a model for populations of finite-state processes with reachability objectives
- We can decide if a controller exists for such a population!

HOWEVER

We assume that there is an opponent moving every process in the worst possible way.

So far...

- We have a model for populations of finite-state processes with reachability objectives
- We can decide if a controller exists for such a population!

HOWEVER

We assume that there is an opponent moving every process in the worst possible way.

- What happens when agents react probabilistically?

Randomised population control [Colcombet Fijalkow Ohlmann '20]

- We start with N processes in the initial state.

Randomised population control [Colcombet Fijalkow Ohlmann '20]

- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$

Randomised population control [Colcombet Fijalkow Ohlmann '20]

- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$
  - Each process selects an x-transition **uniformly at random**

Randomised population control [Colcombet Fijalkow Ohlmann '20]

- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$
  - Each process selects an x-transition **uniformly at random**
- Controller wins when all processes are in the final state.

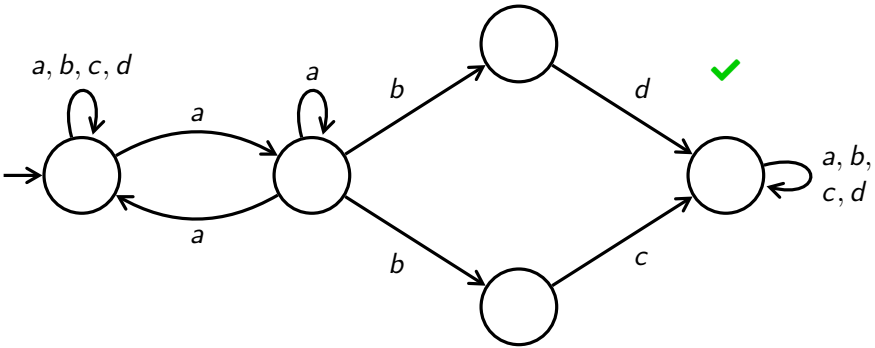
Randomised population control [Colcombet Fijalkow Ohlmann '20]

- We start with N processes in the initial state.
- At each turn:
  - Controller picks a letter $x \in \Sigma$
  - Each process selects an x-transition **uniformly at random**
- Controller wins when all processes are in the final state.

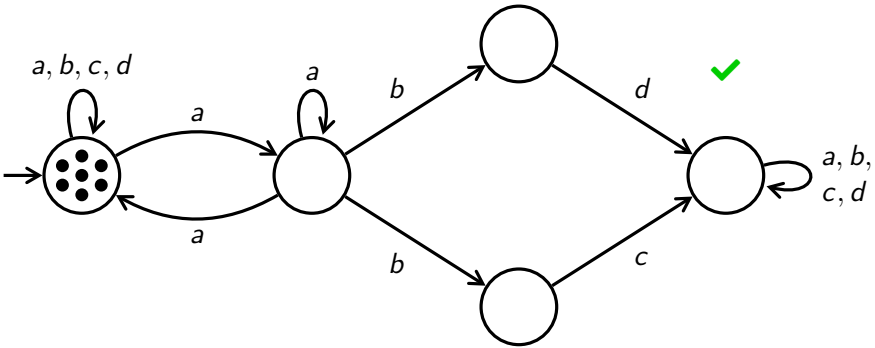
Randomised population control problem

Input: an NFA $\mathcal{A}$
Output: Can Controller win over $\mathcal{A}$ for all N **with probability 1**?

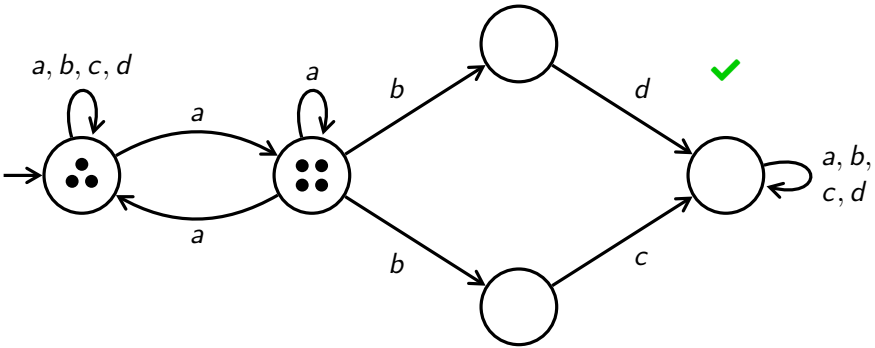
Randomised populations



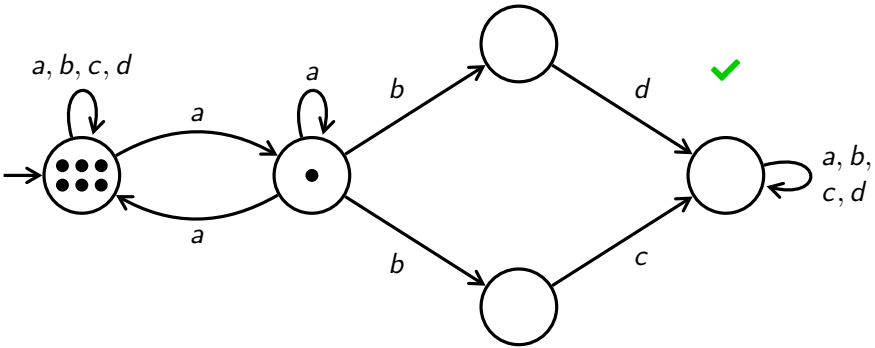
Randomised populations



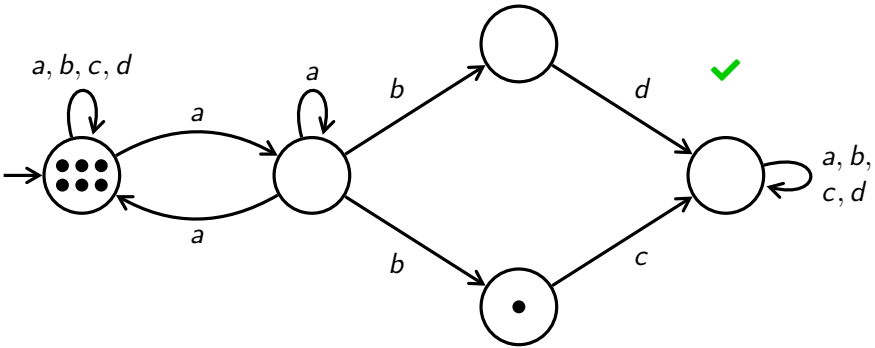
Randomised populations



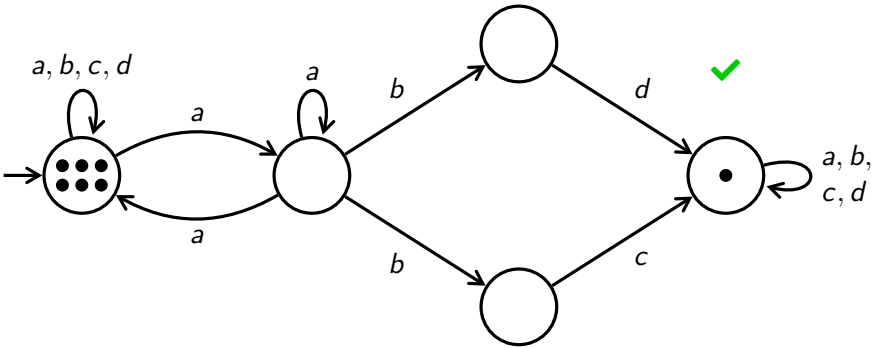
Randomised populations



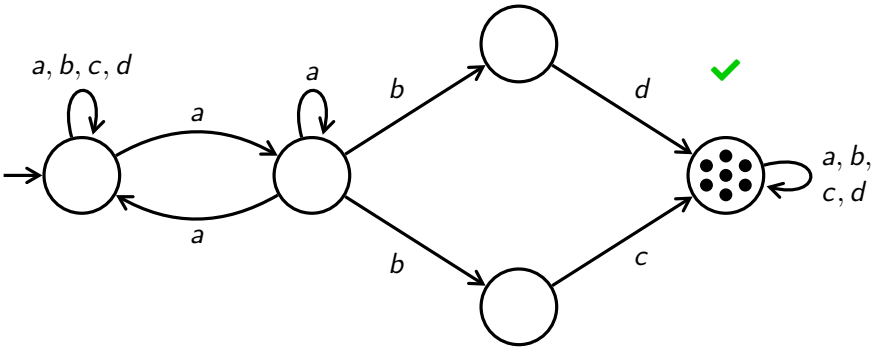
Randomised populations



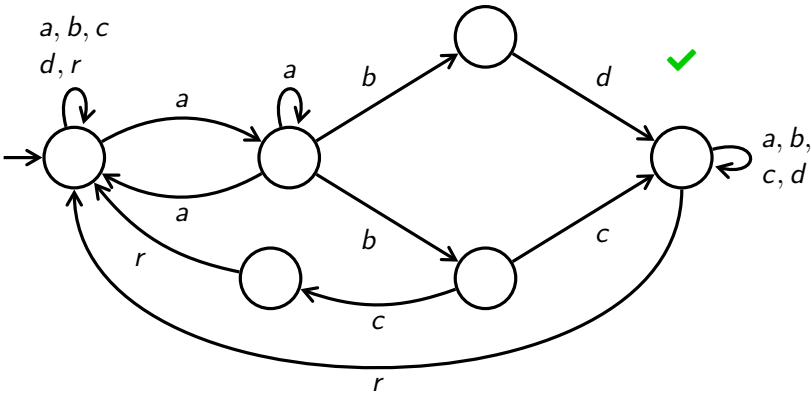
Randomised populations



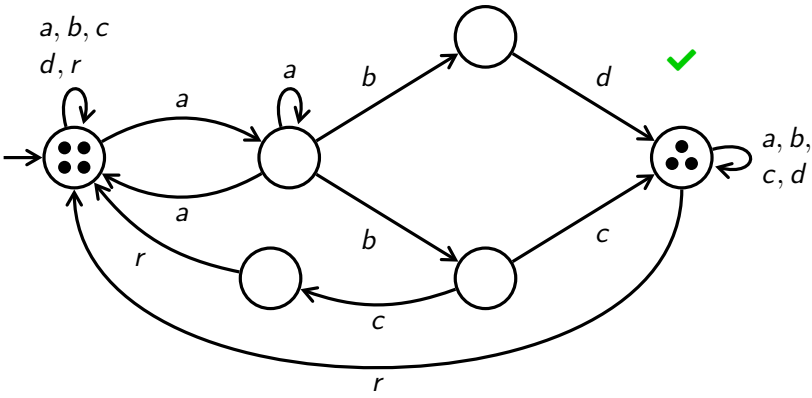
Randomised populations



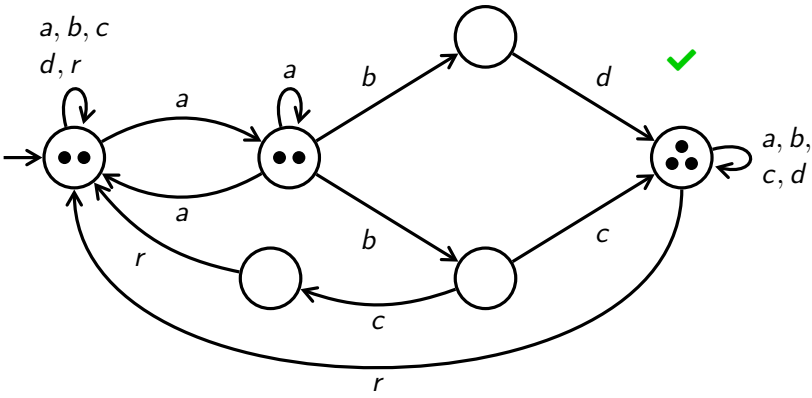
Randomised populations



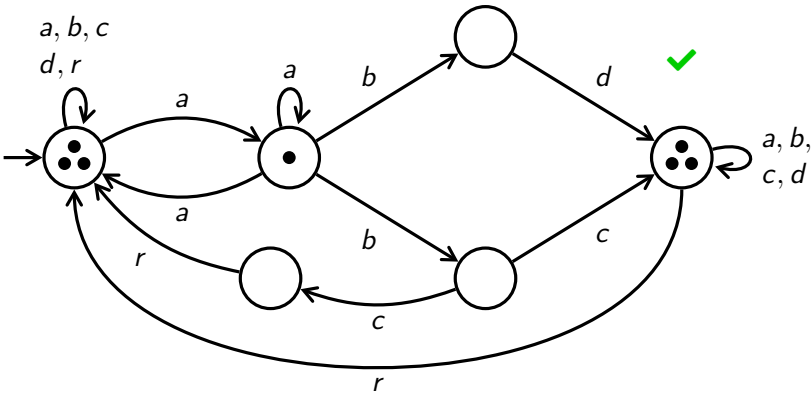
Randomised populations



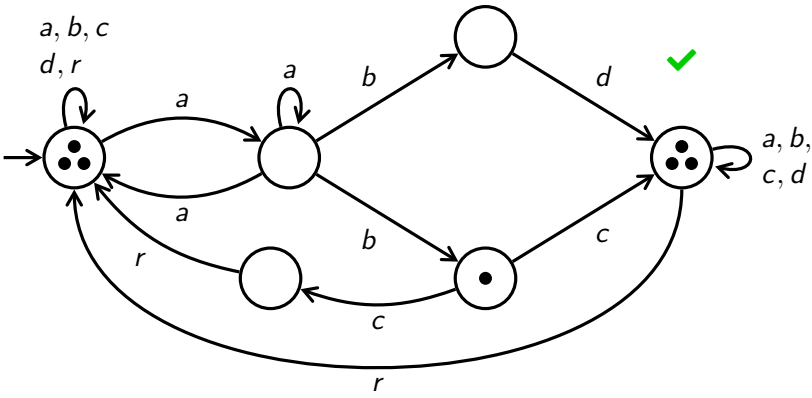
Randomised populations



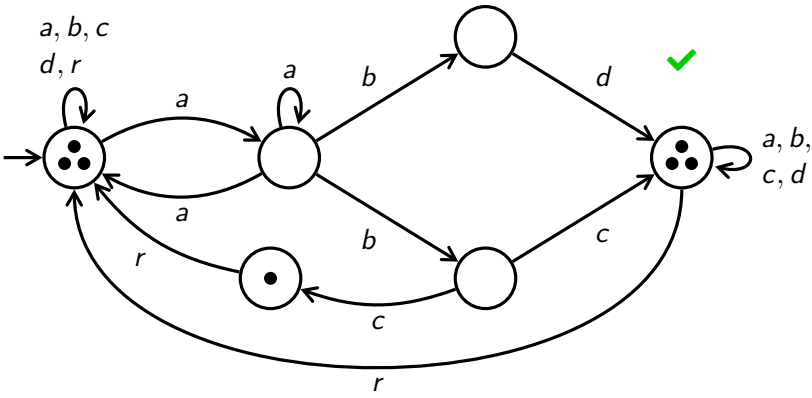
Randomised populations



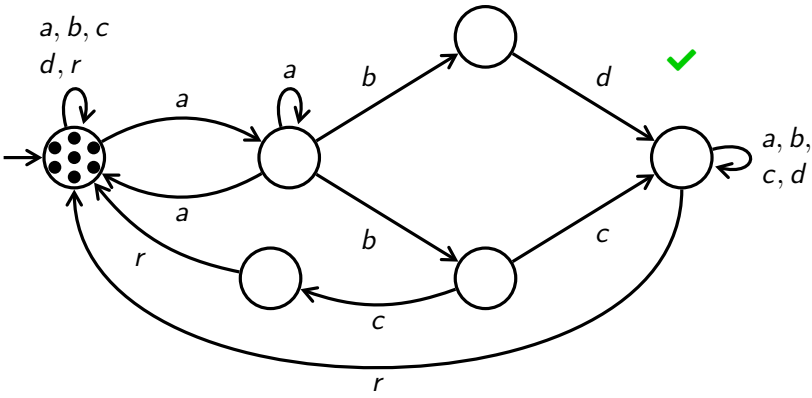
Randomised populations



Randomised populations



Randomised populations



Randomised population control

Theorem [Colcombet Fijalkow Ohlmann '20]

The Randomised population control problem is decidable.

Randomised population control

Theorem [Colcombet Fijalkow Ohlmann '20]

The Randomised population control problem is decidable.

Three ingredients

Randomised population control

Theorem [Colcombet Fijalkow Ohlmann '20]

The Randomised population control problem is decidable.

Three ingredients

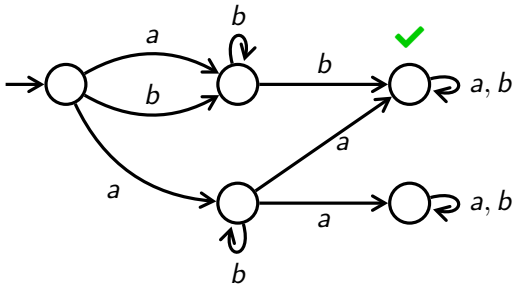
Ingredient I
Winning regions

Ingredient II
Well-quasi-orders

Ingredient III
Sequential flows

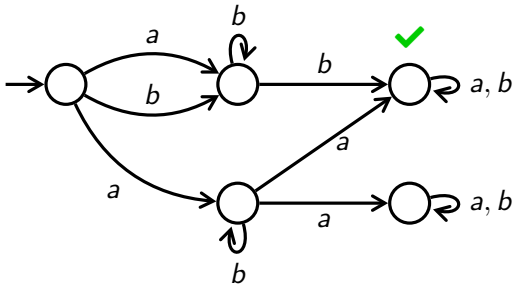
Ingredient I - Winning regions

Let's say we have only one agent.



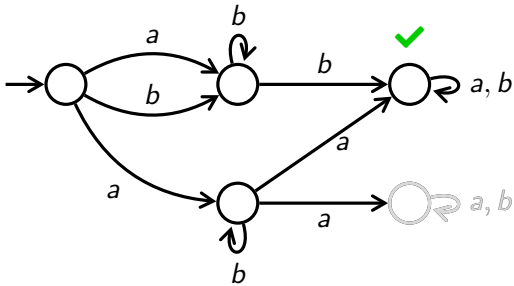
Ingredient I - Winning regions

Let's say we have only one agent.



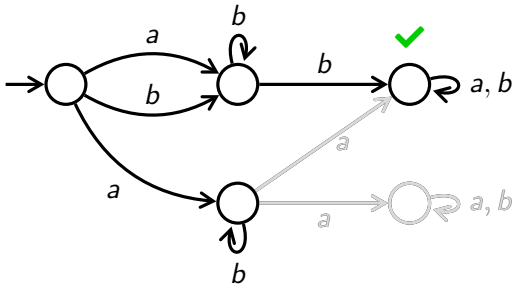
Ingredient I - Winning regions

Let's say we have only one agent.



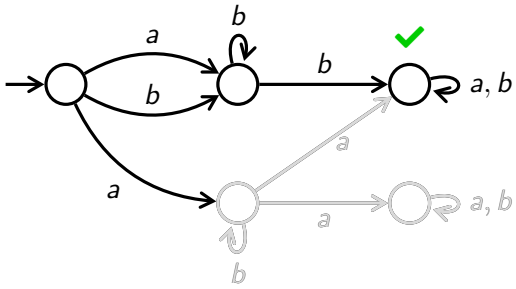
Ingredient I - Winning regions

Let's say we have only one agent.



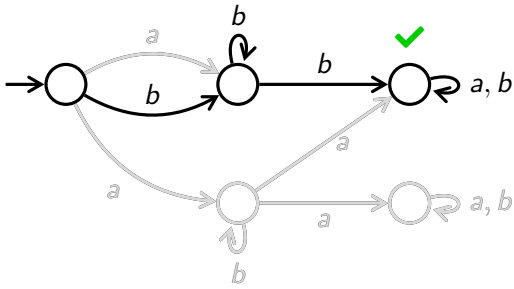
Ingredient I - Winning regions

Let's say we have only one agent.



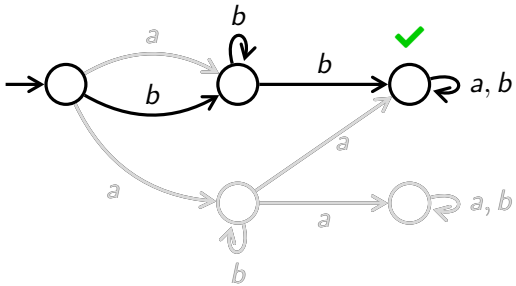
Ingredient I - Winning regions

Let's say we have only one agent.



Ingredient I - Winning regions

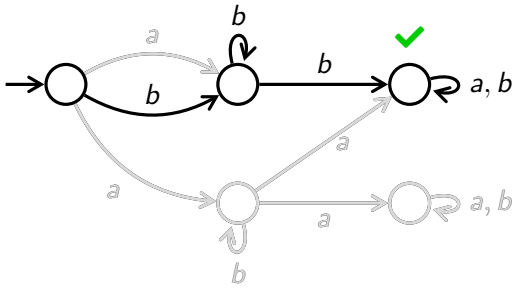
Let's say we have only one agent.



Winning region = set of configurations from which we can reach the target with probability 1.

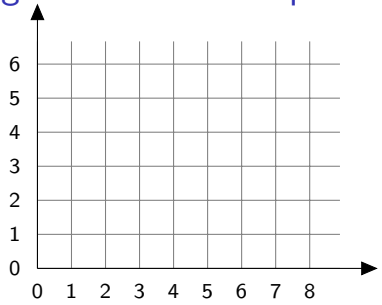
Ingredient I - Winning regions

Let's say we have only one agent.



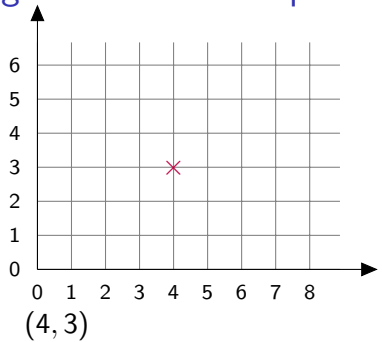
Winning region = set of configurations from which we can reach the target with probability 1.
In general, the winning region is a subset of $\mathbb{N}^S$.

Ingredient II - Well quasi-orders



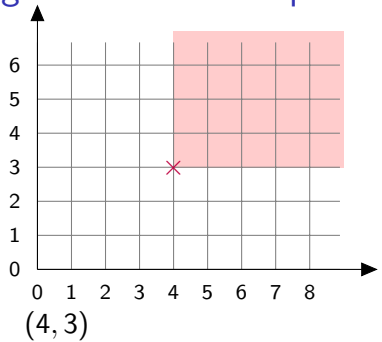
- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Ingredient II - Well quasi-orders



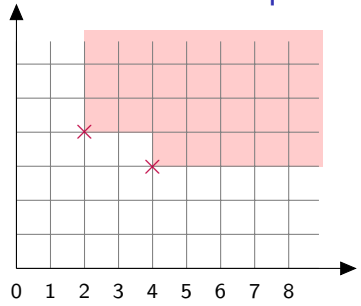
- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Ingredient II - Well quasi-orders



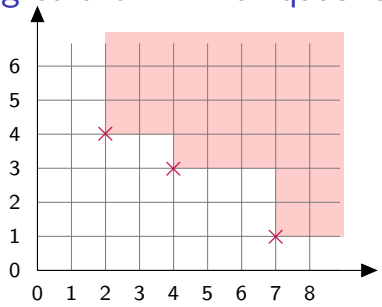
- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Ingredient II - Well quasi-orders



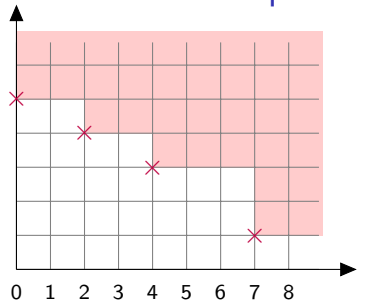
- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Ingredient II - Well quasi-orders



- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

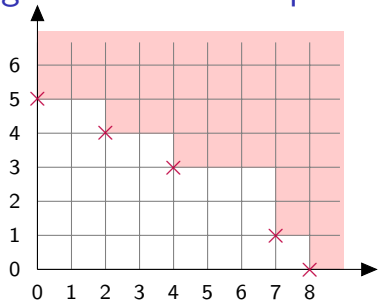
Ingredient II - Well quasi-orders



$(4,3) \to (2,4) \to (7,1) \to (0,5)$

- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

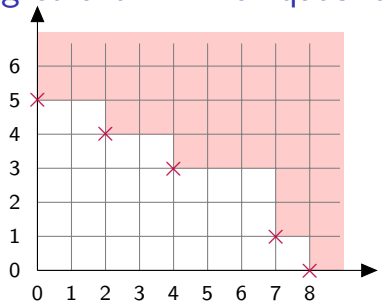
Ingredient II - Well quasi-orders



$(4,3) \to (2,4) \to (7,1) \to (0,5) \to (8,0)$

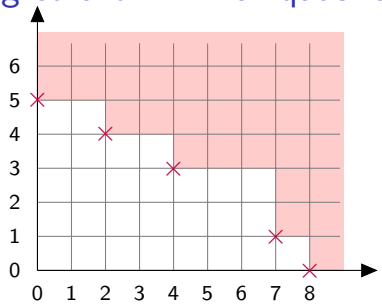
- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Ingredient II - Well quasi-orders



- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

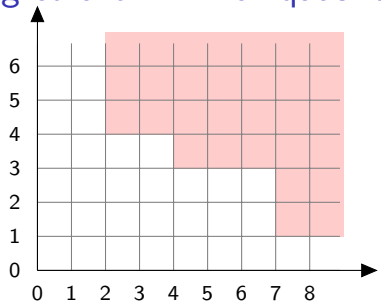
Ingredient II - Well quasi-orders



- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Every set of incomparable elements for the product ordering is finite.

Ingredient II - Well quasi-orders

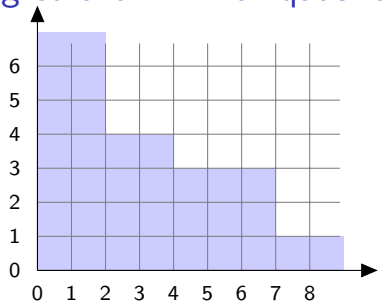


- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Every set of incomparable elements for the product ordering is finite.

Every upward-closed set of $\mathbb{N}^2$ has a finite representation of the form $(2,4)\uparrow \cup (4,3)\uparrow \cup (7,1)\uparrow$.

Ingredient II - Well quasi-orders



- Product ordering: $(x, y) \leq (z, t)$ when $x \leq z$ and $y \leq t$.
- You cannot pick a point higher than one of the previous ones.

Every set of incomparable elements for the product ordering is finite.

Every upward-closed set of $\mathbb{N}^2$ has a finite representation of the form $(2,4)\uparrow \cup (4,3)\uparrow \cup (7,1)\uparrow$.

Every downward-closed set of $\mathbb{N}^2$ has a finite representation of the form $(\omega,1)\downarrow \cup (2,\omega)\downarrow \cup (4,4)\downarrow \cup (7,3)\downarrow$.

Ingredient II - Well quasi-orders

- Fewer tokens is always better - **The winning region is downward-closed.**

Ingredient II - Well quasi-orders

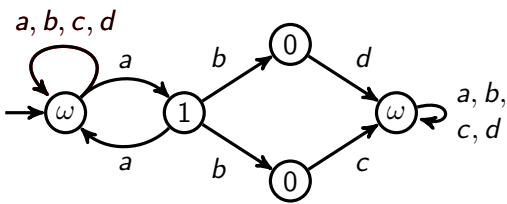
- Fewer tokens is always better - **The winning region is downward-closed.**
- It can be represented as finite unions of *ideals*.

$$(\omega, 5, \omega, 8)\downarrow \cup (4, \omega, \omega, 4)\downarrow \cup (9, 6, 1, 4)\downarrow$$

Ingredient II - Well quasi-orders

- Fewer tokens is always better - **The winning region is downward-closed.**
- It can be represented as finite unions of *ideals*.

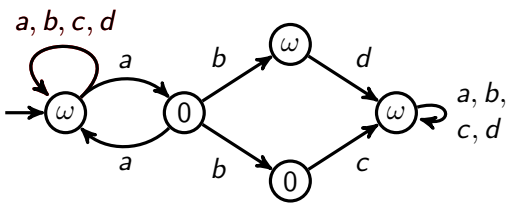
$$(\omega, 5, \omega, 8)\downarrow \cup\ (4, \omega, \omega, 4)\downarrow \cup\ (9, 6, 1, 4)\downarrow$$



Ingredient II - Well quasi-orders

- Fewer tokens is always better - **The winning region is downward-closed.**
- It can be represented as finite unions of *ideals*.

$$(\omega, 5, \omega, 8)\downarrow \cup\ (4, \omega, \omega, 4)\downarrow \cup\ (9, 6, 1, 4)\downarrow$$



Ingredient II - Well quasi-orders

Two semi-decision procedures

For each $N \in \mathbb{N}$:
If there is no strategy against N tokens
Then **NO**

Ingredient II - Well quasi-orders

Two semi-decision procedures

For each $N \in \mathbb{N}$:
If there is no strategy against N tokens
Then **NO**

For each W downward-closed set of configurations and transitions:
If
- W contains all initial configurations,
- All transitions in W are safe, and
- From each configuration in W there is a path to a target configuration within W

Then **YES**

Ingredient III - Back to verification

Set of states S
Tile = function
$S \times S \to \mathbb{N} \cup \{+\infty\}$

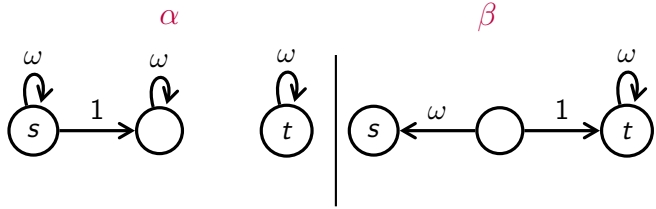
Ingredient III - Back to verification

Set of states S

Tile = function

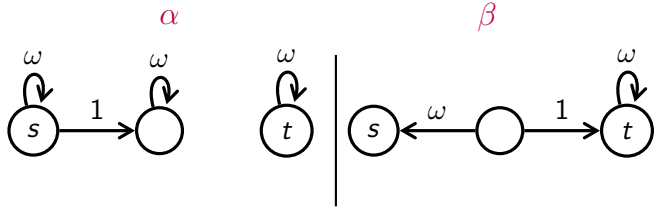
$S \times S \to \mathbb{N} \cup \{+\infty\}$

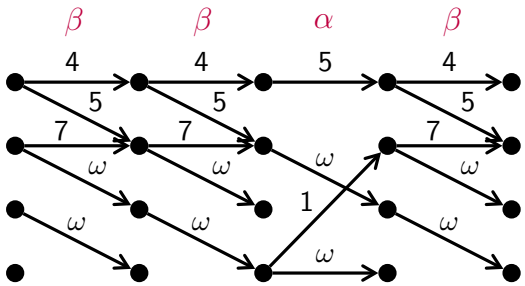
ω = unbounded, no edge = 0.



Ingredient III - Back to verification

Set of states S
Tile = function
$S \times S \to \mathbb{N} \cup \{+\infty\}$
ω = unbounded, no edge = 0.





Problem [Colcombet, Fijalkow, Ohlmann '20]

Input: A set of tiles
Output: Maximum number of agents we can transfer from s to t?

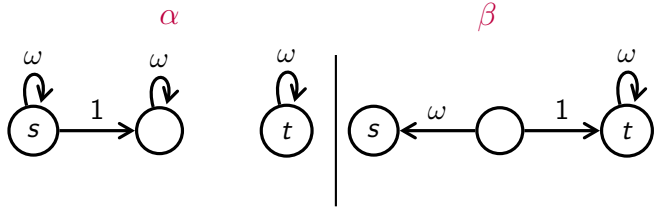
Ingredient III - Back to verification

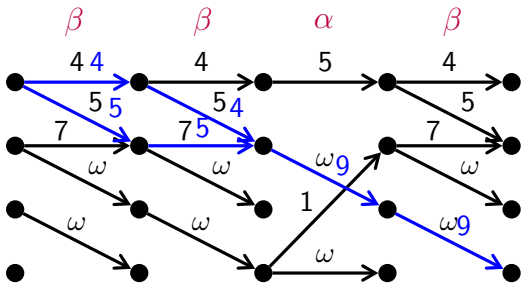
Set of states S

Tile = function

$S \times S \to \mathbb{N} \cup \{+\infty\}$

ω = unbounded, no edge = 0.





Problem [Colcombet, Fijalkow, Ohlmann '20]

Input: A set of tiles

Output: Maximum number of agents we can transfer from s to t?

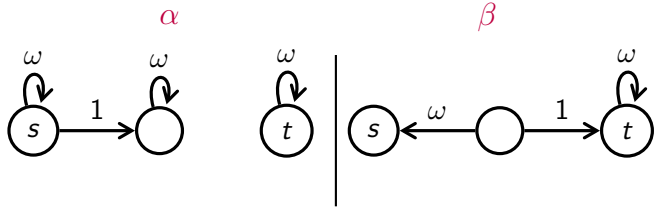
Ingredient III - Back to verification

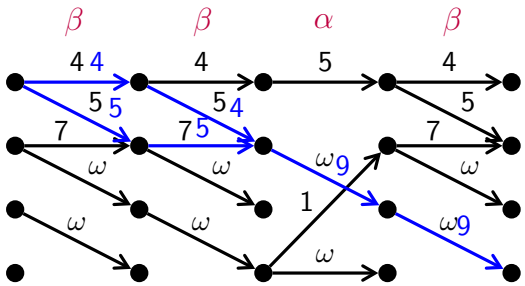
Set of states S

Tile = function

$S \times S \to \mathbb{N} \cup \{+\infty\}$

ω = unbounded, no edge = 0.





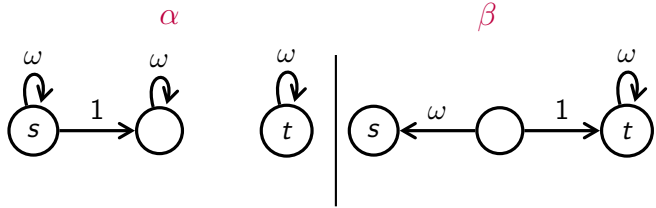
Problem [Colcombet, Fijalkow, Ohlmann '20]

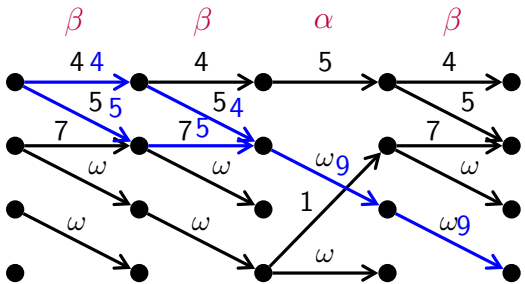
Input: A set of tiles

Output: Maximum number of agents we can transfer from s to t?

Ingredient III - Back to verification

Set of states S
Tile = function
$S \times S \to \mathbb{N} \cup \{+\infty\}$
ω = unbounded, no edge = 0.





Problem [Colcombet, Fijalkow, Ohlmann '20]

Input: A set of tiles
Output: Maximum number of agents we can transfer from s to t?

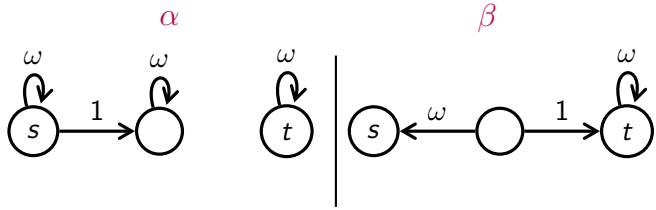
Transfer of tokens = Flow!

Ingredient III - Sequential flows

Set of states S
Tile = function
$S \times S \to \mathbb{N} \cup \{+\infty\}$

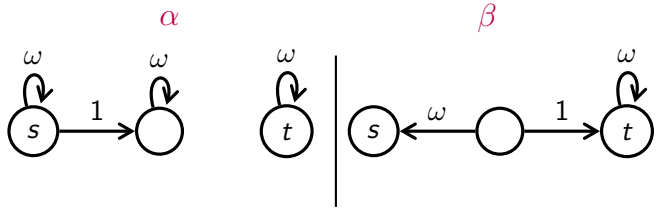
Ingredient III - Sequential flows

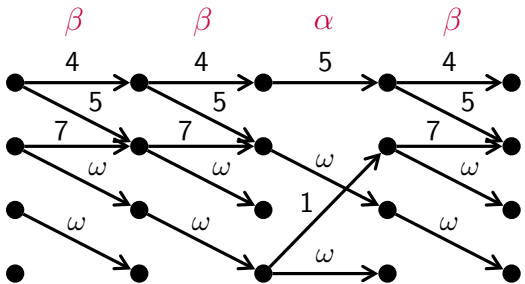
Set of states S
Tile = function
$S \times S \to \mathbb{N} \cup \{+\infty\}$
ω = unbounded, no edge = 0.



Ingredient III - Sequential flows

Set of states S
Tile = function
$S \times S \to \mathbb{N} \cup \{+\infty\}$
ω = unbounded, no edge = 0.



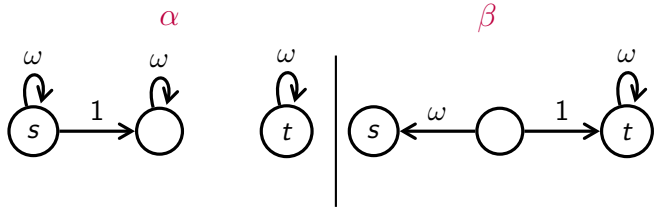


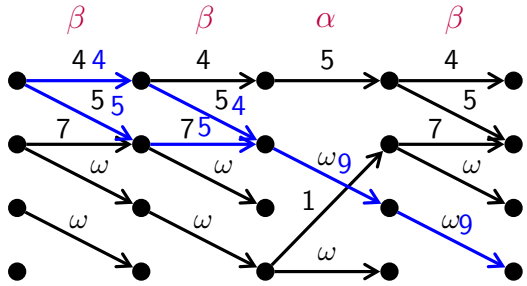
Problem [Colcombet, Fijalkow, Ohlmann '20]

Input: A set of tiles
Output: Maximum number of agents we can transfer from s to t?

Ingredient III - Sequential flows

Set of states S
Tile = function
$S \times S \to \mathbb{N} \cup \{+\infty\}$
ω = unbounded, no edge = 0.

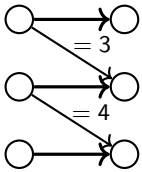




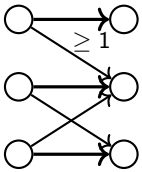
Problem [Colcombet, Fijalkow, Ohlmann '20]

Input: A set of tiles
Output: Maximum number of agents we can transfer from s to t?

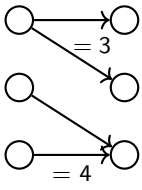
Tiles and flows



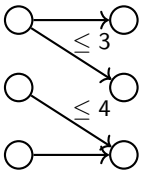
≈ Petri nets



≈Shared variables



≈Counter systems



≈ Locks

Sequential Flow Problem [Colcombet, Fijalkow, Ohlmann '20]

Problem

Input: *A set of tiles* **Tiles**

Output: *Is* $\{\texttt{MaxFlow}(w) \mid w \in \textbf{Tiles}^*\}$ *unbounded?*

Sequential Flow Problem [Colcombet, Fijalkow, Ohlmann '20]

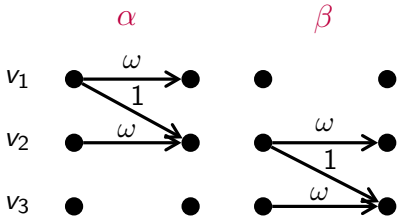
Problem

Input: *A set of tiles* **Tiles**
Output: *Is* $\{\texttt{MaxFlow}(w) \mid w \in \textbf{Tiles}^*\}$ *unbounded?*

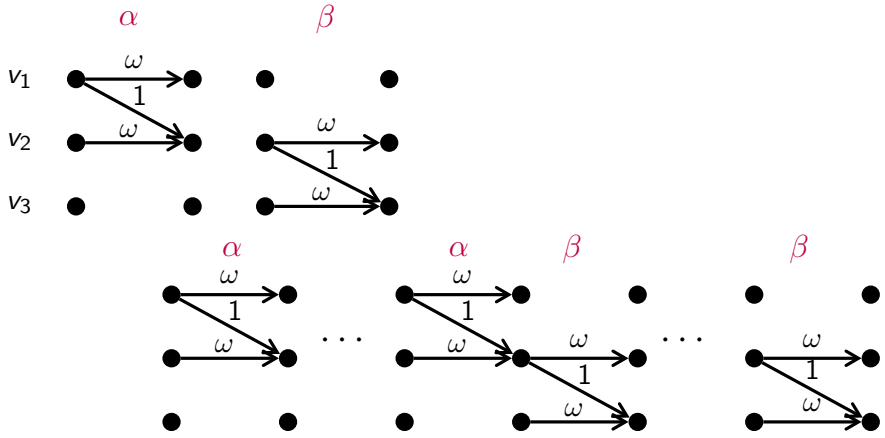
Problem

Input: *A set of tiles* **Tiles**
Output: *Compute* $\sup\{\texttt{MaxFlow}(w) \mid w \in \textbf{Tiles}^*\}$.

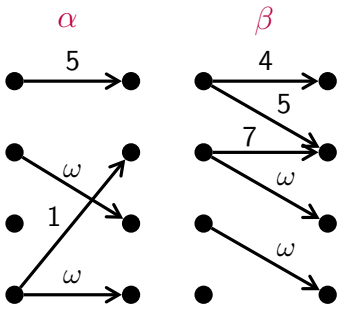
Example



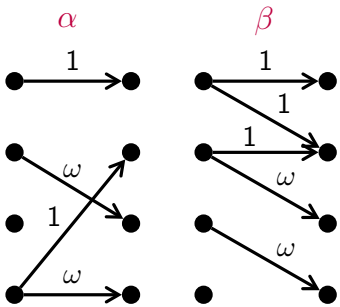
Example



Abstraction

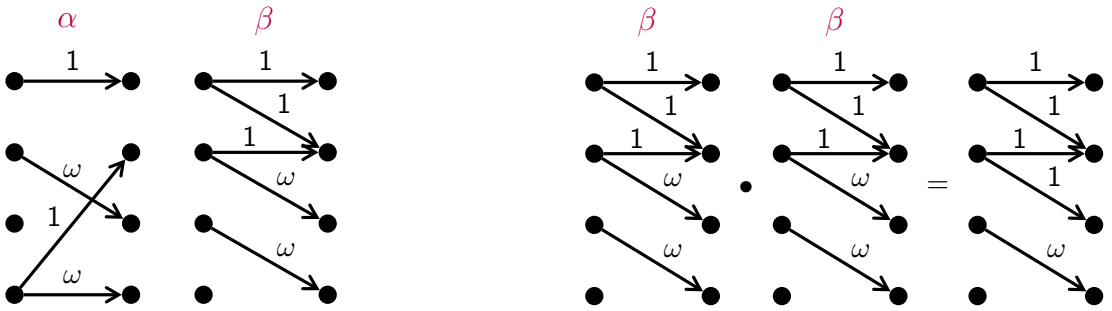


Abstraction



Morphism $\varphi : \mathbf{Tiles}^* \to \{0, 1, \omega\}^{V \times V}$ with max-min product.

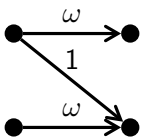
Abstraction



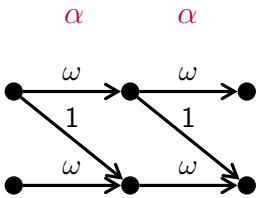
Morphism $\varphi : \mathbf{Tiles}^* \to \{0, 1, \omega\}^{V \times V}$ with max-min product.

Iteration (Inspired by work of Simon, Hashigushi, Leung)

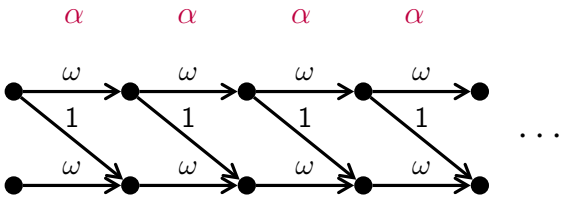
α



Iteration (Inspired by work of Simon, Hashigushi, Leung)

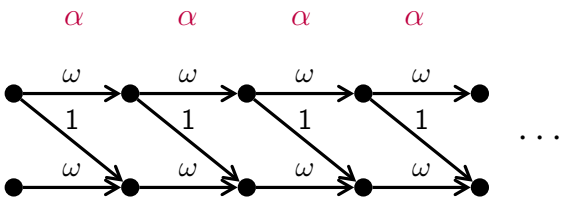


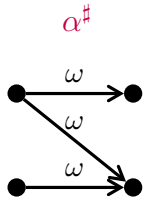
Iteration (Inspired by work of Simon, Hashigushi, Leung)



$\alpha^N = \alpha$ (α is *idempotent*) but α^N has flow N.

Iteration (Inspired by work of Simon, Hashigushi, Leung)



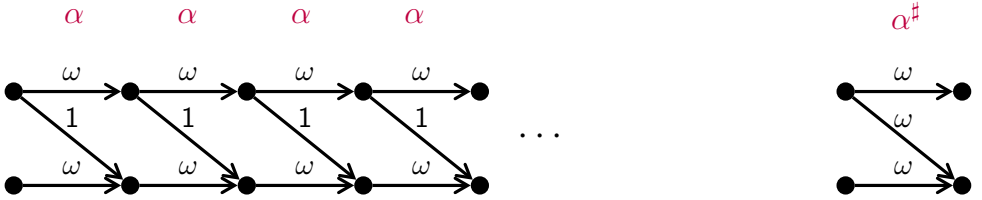


$\alpha^N = \alpha$ (α is *idempotent*) but α^N has flow N.

Define a new operator $\sharp$ on idempotents:

$$\alpha^\sharp(s,t) = \text{effect of } \alpha \text{ iterated many many times.}$$

Iteration (Inspired by work of Simon, Hashigushi, Leung)



$\alpha^N = \alpha$ (α is *idempotent*) but α^N has flow N.

Define a new operator $\sharp$ on idempotents:

$$\alpha^\sharp(s,t) = \text{effect of } \alpha \text{ iterated many many times.}$$

If $\alpha^\sharp(s,t) = \omega$ then we have unbounded flows between s and t.

Abstraction

$\mathcal{F}_\sharp$ the closure of $\varphi(\textbf{Tiles})$ under product and $\sharp$.

Abstraction

$\mathcal{F}_\sharp$ the closure of $\varphi(\textbf{Tiles})$ under product and $\sharp$.

Lemma

If $\mathcal{F}_\sharp$ contains some α with $\alpha(s,t) = \omega$ then there are unbounded flows between s and t.

Abstraction

$\mathcal{F}_\sharp$ the closure of $\varphi(\textbf{Tiles})$ under product and $\sharp$.

Lemma

If $\mathcal{F}_\sharp$ contains some α with $\alpha(s,t) = \omega$ then there are unbounded flows between s and t.

What about the other direction?

Factorisation forest theorem [Simon '90]

Σ finite alphabet, $(\boldsymbol{M}, \cdot)$ finite monoid, $\varphi : \Sigma^* \to \mathbf{M}$ morphism.

Factorisation forest theorem [Simon '90]

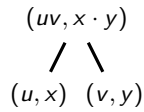
Σ finite alphabet, $(\boldsymbol{M}, \cdot)$ finite monoid, $\varphi : \Sigma^* \to \mathbf{M}$ morphism.
Factorisation tree: Node labels in $\Sigma^* \times \boldsymbol{M}$.

3 types of nodes:

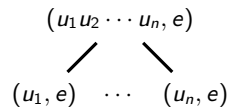
Leaves

$(a, \varphi(a))$

Product nodes



Idempotent nodes



Factorisation forest theorem [Simon '90]

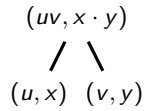
Σ finite alphabet, $(\boldsymbol{M}, \cdot)$ finite monoid, $\varphi : \Sigma^* \to \mathbf{M}$ morphism.
Factorisation tree: Node labels in $\Sigma^* \times \boldsymbol{M}$.

3 types of nodes:

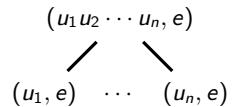
Leaves

$(a, \varphi(a))$

Product nodes



Idempotent nodes



Theorem

Every word has a factorisation tree of height $\leq 3|\mathbf{M}|$

Abstraction

$\mathcal{F}_\sharp$ the closure of $\varphi(\textbf{Tiles})$ under product and $\sharp$.

Abstraction

$\mathcal{F}_\sharp$ the closure of $\varphi(\textbf{Tiles})$ under product and $\sharp$.

Lemma

$\mathcal{F}_\sharp$ contains some α with $\alpha(s,t) = \omega$ if and only if there are unbounded flows between s and t.

Proof by induction on the height of factorization trees.

Abstraction

$\mathcal{F}_\sharp$ the closure of $\varphi(\textbf{Tiles})$ under product and $\sharp$.

Lemma

$\mathcal{F}_\sharp$ contains some α with $\alpha(s,t) = \omega$ if and only if there are unbounded flows between s and t.

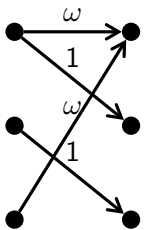
Proof by induction on the height of factorization trees.

But how large can a bounded flow be in terms of nb of vertices n?

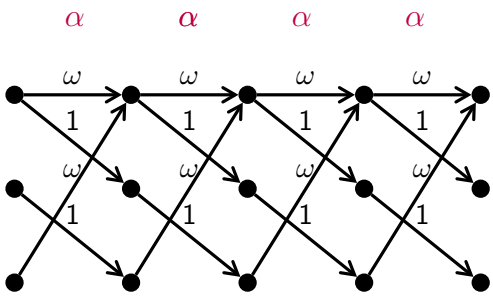
Using Simon's theorem $\rightarrow 2^{3|\mathcal{F}_\#|} \leq 2^{3^{n^2+1}}$
Can we do better?

Iteration dichotomy

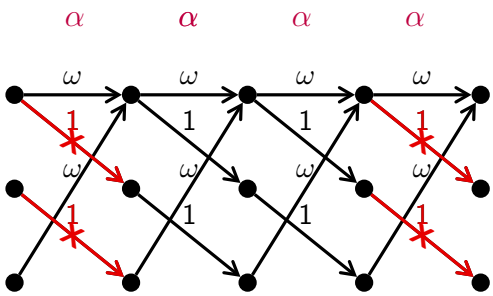
α



Iteration dichotomy



Iteration dichotomy



For all idempotent α and $s, t \in Q$,

- either $\alpha^\sharp(s,t) = \omega$
- or there is a bounded *cut* between s and t in the first and last α.

Short-ends factorisation theorem (based on work by [Simon '90], [Colcombet '11])

Σ finite alphabet, $(\boldsymbol{M}, \cdot)$ finite monoid, $\varphi : \Sigma^* \to \mathbf{M}$ morphism.

Short-ends factorisation theorem (based on work by [Simon '90], [Colcombet '11])

Σ finite alphabet, $(\boldsymbol{M}, \cdot)$ finite monoid, $\varphi : \Sigma^* \to \mathbf{M}$ morphism.

Short-ends factorisation tree:

Leaves

$(a, \varphi(a))$

Product nodes

$(uv, x \cdot y)$

(u, x) (v, y)

Idempotent nodes

$(u_1 u_2 \cdots u_n, e)$

(u_1, e) (u_n, e)

$\varphi(u_i) = e$ for all i.

Ramsey bounds (based on work by [Jecker '21])

$R_{\mathbf{M}}(k)$ = minimal n such that every word w of length n has a factor $u_1 \dots u_k$ with $\varphi(u_1) = \cdots = \varphi(u_k) = e$ an idempotent of $\mathbf{M}$.

Ramsey bounds (based on work by [Jecker '21])

$R_{\mathbf{M}}(k)$ = minimal n such that every word w of length n has a factor $u_1 \dots u_k$ with $\varphi(u_1) = \cdots = \varphi(u_k) = e$ an idempotent of $\mathbf{M}$.

Theorem

For all $w \in \Sigma^*$, there is a short-ends factorisation of height $poly(log(\mathbf{M}), log(R_{\mathbf{M}}(3)))$.

Corollary

In a transition monoid $\{\top, \bot\}^{n \times n}$, every word has a short-ends factorisation of height $poly(n)$.

Exponential bound

- Every word has a factorisation of polynomial height in n.
- If there is a bounded cut between s and t in a sequence of tiles, then there is one that lives within the factorisation.

Theorem

If the flow between s and t is bounded then it is at most exponential in n.

Results

Theorem [Colcombet, Fijalkow, Ohlmann '20]

The Randomised population control problem is decidable.

Results

Theorem [Colcombet, Fijalkow, Ohlmann '20]

The Randomised population control problem is decidable.

Theorem [Gimbert, M., Totzke '26]

The Randomised population control problem is EXPTIME-complete.

Sequential Flow Problem (SFP)

[Blumensath Colcombet Parys '16]

Satisfiability in some extension of MSO reduces to SFP*

[Blumensath Colcombet Parys '16]

SFP* is decidable.

Sequential Flow Problem (SFP)

[Blumensath Colcombet Parys '16]

Satisfiability in some extension of MSO reduces to SFP*

[Colcombet Fijalkow Ohlmann '20]

The Randomised Population Control Problem reduces to SFP.

[Blumensath Colcombet Parys '16]

SFP* is decidable.

[Colcombet Fijalkow Ohlmann '20]

SFP is PSPACE-hard and in EXPSPACE.

Sequential Flow Problem (SFP)

[Blumensath Colcombet Parys '16]

Satisfiability in some extension of MSO reduces to SFP*

[Colcombet Fijalkow Ohlmann '20]

The Randomised Population Control Problem reduces to SFP.

[Gimbert Mascle Totzke '26b]

The Randomised Population Control Problem reduces to SFP in exponential time.

[Blumensath Colcombet Parys '16]

SFP* is decidable.

[Colcombet Fijalkow Ohlmann '20]

SFP is PSPACE-hard and in EXPSPACE.

[Gimbert Mascle Totzke '26a]

SFP is PSPACE-complete.

Sequential Flow Problem (SFP)

[Blumensath Colcombet Parys '16]

Satisfiability in some extension of MSO reduces to SFP*

[Blumensath Colcombet Parys '16]

SFP* is decidable.

[Colcombet Fijalkow Ohlmann '20]

The Randomised Population Control Problem reduces to SFP.

[Colcombet Fijalkow Ohlmann '20]

SFP is PSPACE-hard and in EXPSPACE.

[Gimbert Mascle Totzke '26b]

The Randomised Population Control Problem reduces to SFP in exponential time.

+

[Gimbert Mascle Totzke '26a]

SFP is PSPACE-complete.

Theorem ([Gimbert Mascle Totzke '26b])

The Randomised Population Control Problem is EXPTIME-complete.

First steps in modeling population control problems

Theorem ([Bertrand Dewaskar Genest Gimbert '17])

The Adversarial Population Control Problem is EXPTIME-complete.

Theorem ([Gimbert Mascle Totzke '26b])

The Randomised Population Control Problem is EXPTIME-complete.

Related work

Explorability for automata on infinite words
[Hazard, Kuperberg, Idir '25], [Bedi, Lehtinen '26]

Related work

Explorability for automata on infinite words
[Hazard, Kuperberg, Idir '25], [Bedi, Lehtinen '26]

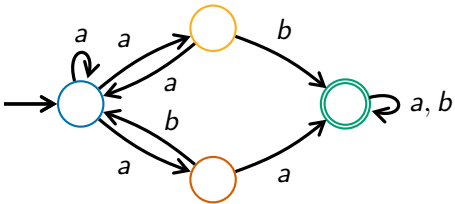
Distribution-based models - continuous distribution instead of discrete population
[Akshay, Genest, Vyas '18], [Doyen, van den Bogaart '22]

Future work

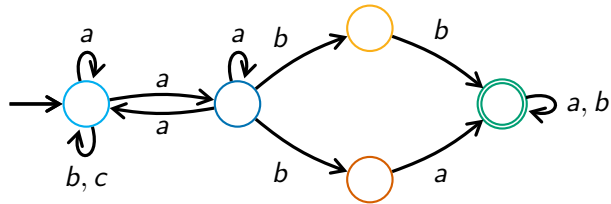
- Quantitative objectives
- Infinite-horizon specifications
- Weighted states/actions
- Continuous state space / time
- Restricted classes of automata

Future work

- Quantitative objectives
- Infinite-horizon specifications
- Weighted states/actions



- Continuous state space / time
- Restricted classes of automata
- **Fast strategies**



Future work

- Quantitative objectives
- Infinite-horizon specifications
- Weighted states/actions
- Continuous state space / time
- Restricted classes of automata
- **Fast strategies**

Refining the population model

Allow agents to have different objectives

Allow agents to have different distributions

Allow influence between agents

Future work

- Quantitative objectives
- Infinite-horizon specifications
- Weighted states/actions
- Continuous state space / time
- Restricted classes of automata
- **Fast strategies**

Refining the population model

Allow agents to have different objectives

Allow agents to have different distributions

Allow influence between agents

Thank you for your attention!